AlerteZone

Explication des diagrammes de conception

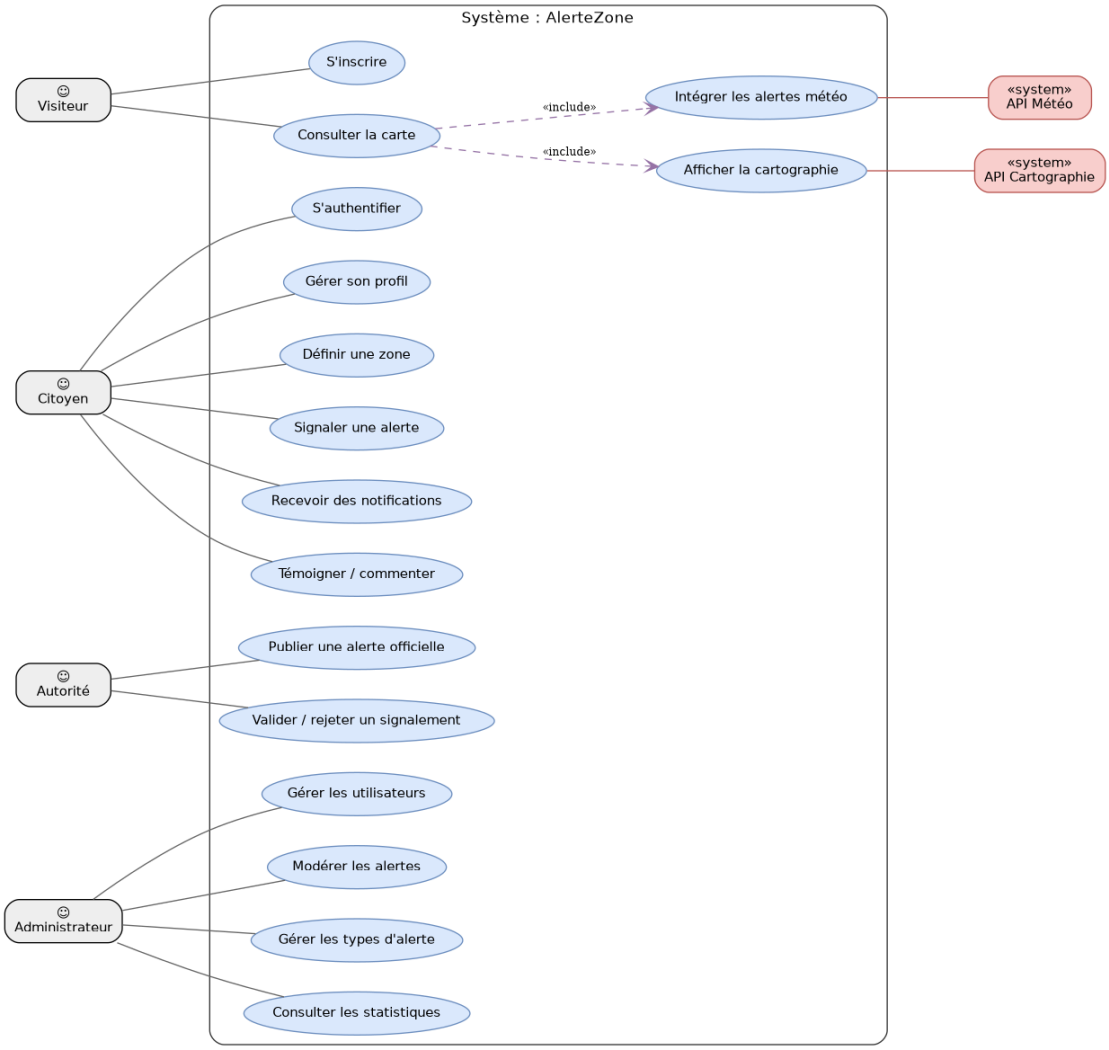
Projet fil rouge — Concepteur Développeur d'Applications (IPSSI, 3■ année)

Alain-Patrick Gomas

Ce document présente chaque diagramme du projet : à quoi il sert, l'image du diagramme, et les points les plus importants à retenir. Il couvre la modélisation de la base de données (méthode MERISE, jalon 3) et la conception de l'application (UML + architecture, jalon 4).

1. Diagramme de cas d'utilisation

Jalon 4 — UML · Montre qui (les acteurs) peut faire quoi (les fonctionnalités). Vue d'ensemble du périmètre.



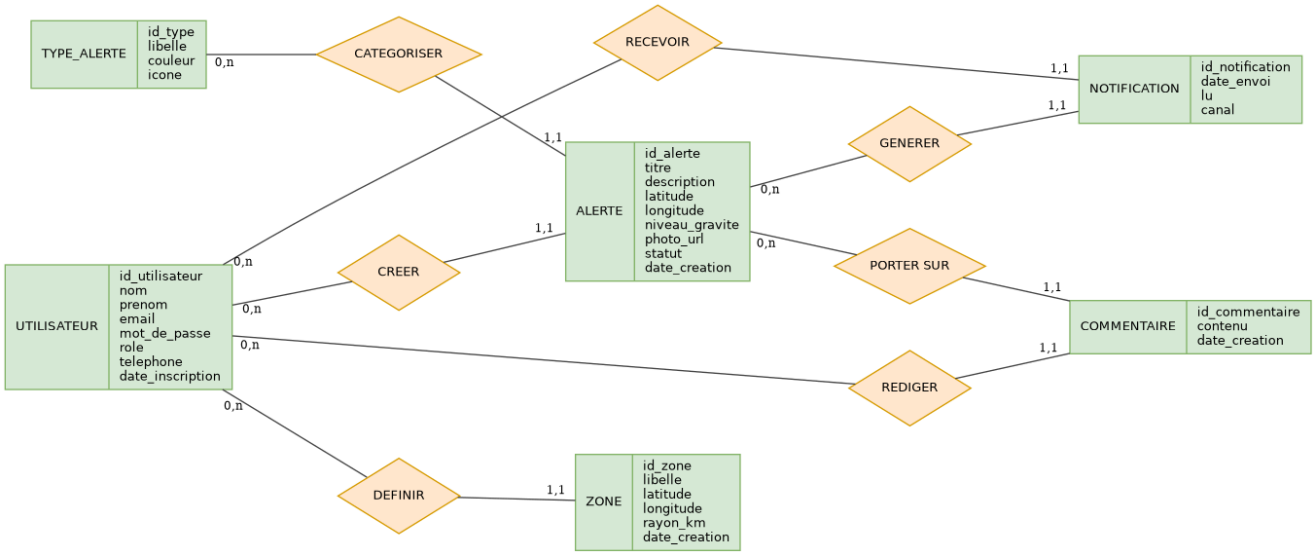
Les points les plus importants

- **4 acteurs humains** : Visiteur (non connecté), Citoyen, Autorité, Administrateur — chacun a ses droits propres.
- **2 systèmes externes** : l'API Météo (OpenWeatherMap) et l'API Cartographie alimentent l'application.
- Le **Citoyen** est au cœur : il signale, consulte la carte, définit sa zone, reçoit des notifications, témoigne.
- L'**Autorité** publie les alertes officielles (disparition, avis de recherche) et valide les signalements citoyens.
- Les liens «include» montrent qu'afficher la carte réutilise la cartographie et les alertes météo.

À dire à l'oral : « Chaque fonctionnalité de mon cahier des charges est rattachée à au moins un acteur : je vérifie ainsi que rien n'est oublié. »

2. MCD — Modèle Conceptuel de Données

Jalon 3 — MERISE · Le plan conceptuel de la base : les grandes notions (entités) et leurs liens, sans technique.



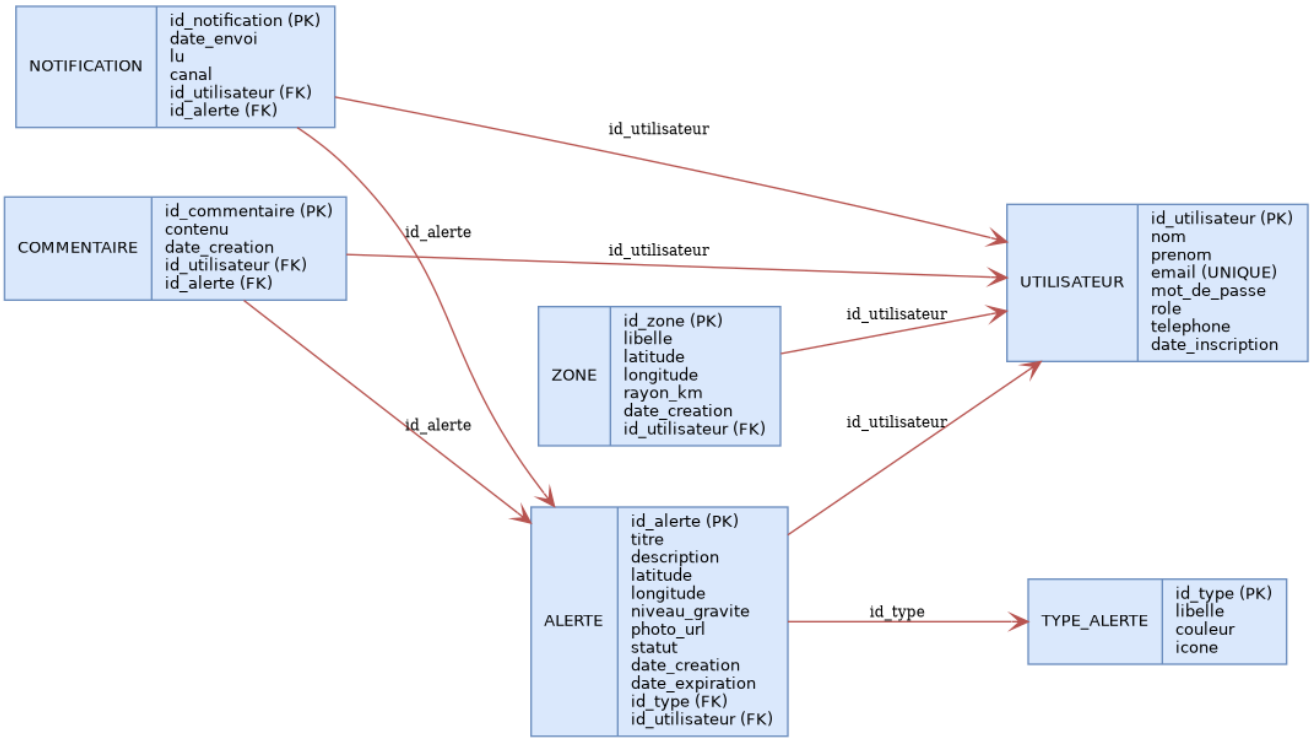
Les points les plus importants

- **6 entités** : Utilisateur, Zone, Type_Alerte, Alerte, Notification, Commentaire.
- Les **losanges** sont les associations (CREER, DEFINIR, CATEGORISER, GENERER…).
- Les **cardinalités** (0,n / 1,1) donnent les règles : un utilisateur crée plusieurs alertes, une alerte a un seul auteur.
- Une alerte a un seul type ; un type concerne plusieurs alertes.
- Notification et Commentaire relient toujours un Utilisateur ET une Alerte.

À dire à l'oral : « Le MCD, c'est les idées : les objets du métier et leurs relations, avant toute technique. »

3. MLD — Modèle Logique de Données

Jalon 3 — MERISE · La traduction du MCD en tables, avec clés primaires (PK) et clés étrangères (FK).



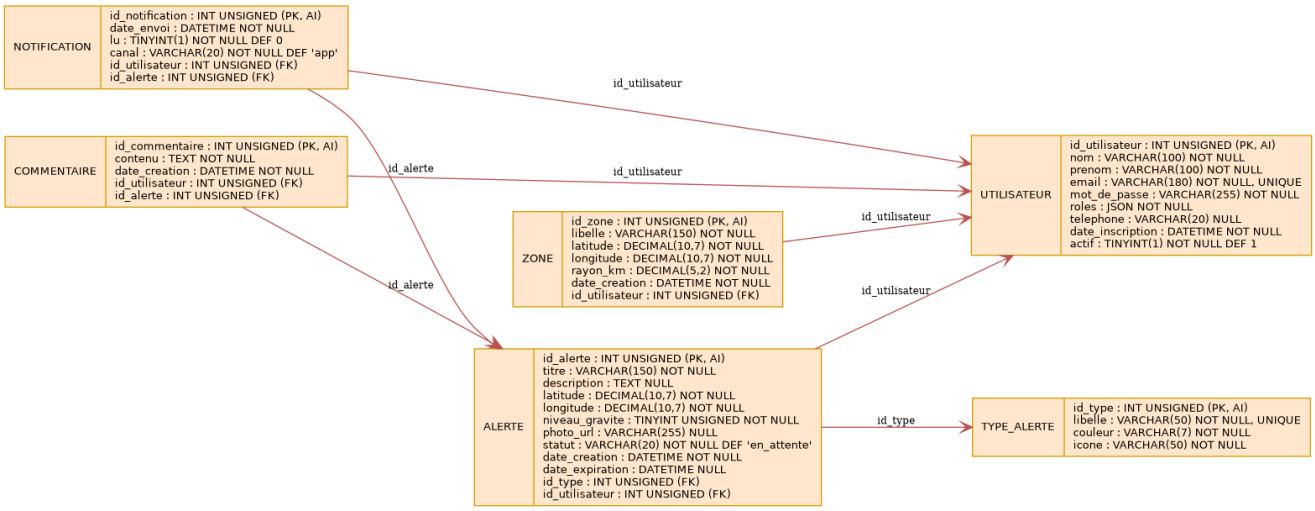
Les points les plus importants

- Chaque **entité devient une table** ; chaque association devient une **clé étrangère**.
- La table ALERTE porte 2 FK : **id_type** et **id_utilisateur**.
- NOTIFICATION et COMMENTAIRE portent chacune 2 FK (vers Utilisateur et Alerte).
- **email** est en UNIQUE : impossible d'avoir deux comptes avec le même email.
- Toutes les entités du MCD se retrouvent en tables : rien n'est perdu.

À dire à l'oral : « Le MLD, c'est le MCD transformé en tables reliées par des clés. »

4. MPD — Modèle Physique de Données

Jalon 3 — MERISE · Le schéma prêt pour MySQL : types réels, index, règles de suppression.



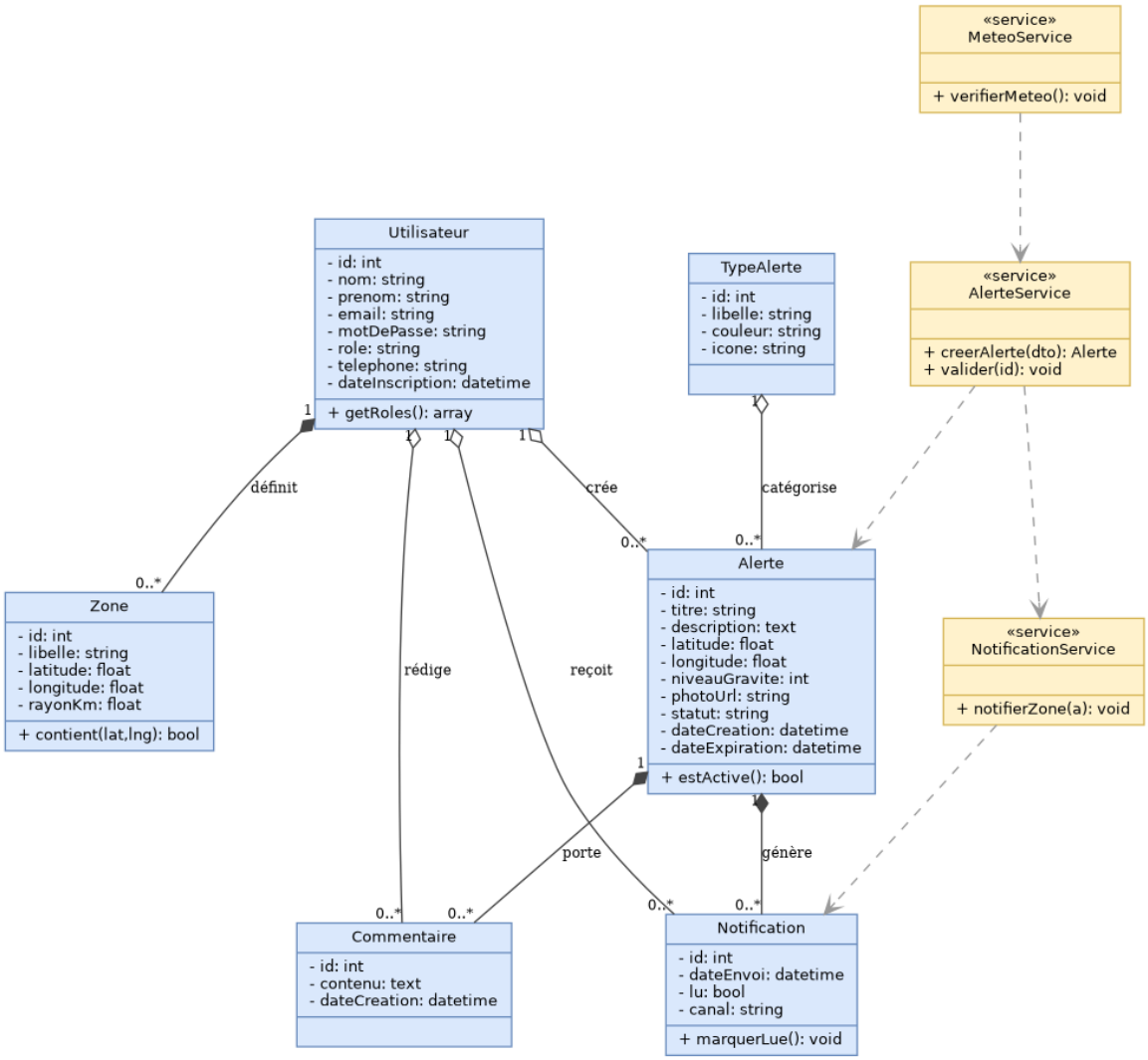
Les points les plus importants

- **Types concrets MySQL** : INT UNSIGNED AUTO_INCREMENT (clés), DECIMAL(10,7) (GPS), DATETIME, TEXT, JSON (rôles Symfony).
- **Index** sur les clés étrangères et les colonnes souvent filtrées (statut, position GPS).
- **ON DELETE CASCADE** : supprimer un utilisateur supprime ses données — utile pour le RGPD.
- Moteur **InnoDB** (gère les clés étrangères) + encodage **utf8mb4** (accents et emojis).
- Ce modèle correspond exactement au script SQL CREATE TABLE fourni avec le projet.

À dire à l'oral : « Le MPD, c'est le MLD adapté à MySQL, avec les types réels, les index et les suppressions en cascade. »

5. Diagramme de classes

Jalon 4 — UML · La structure du code côté application : les classes, leurs attributs et leurs liens.



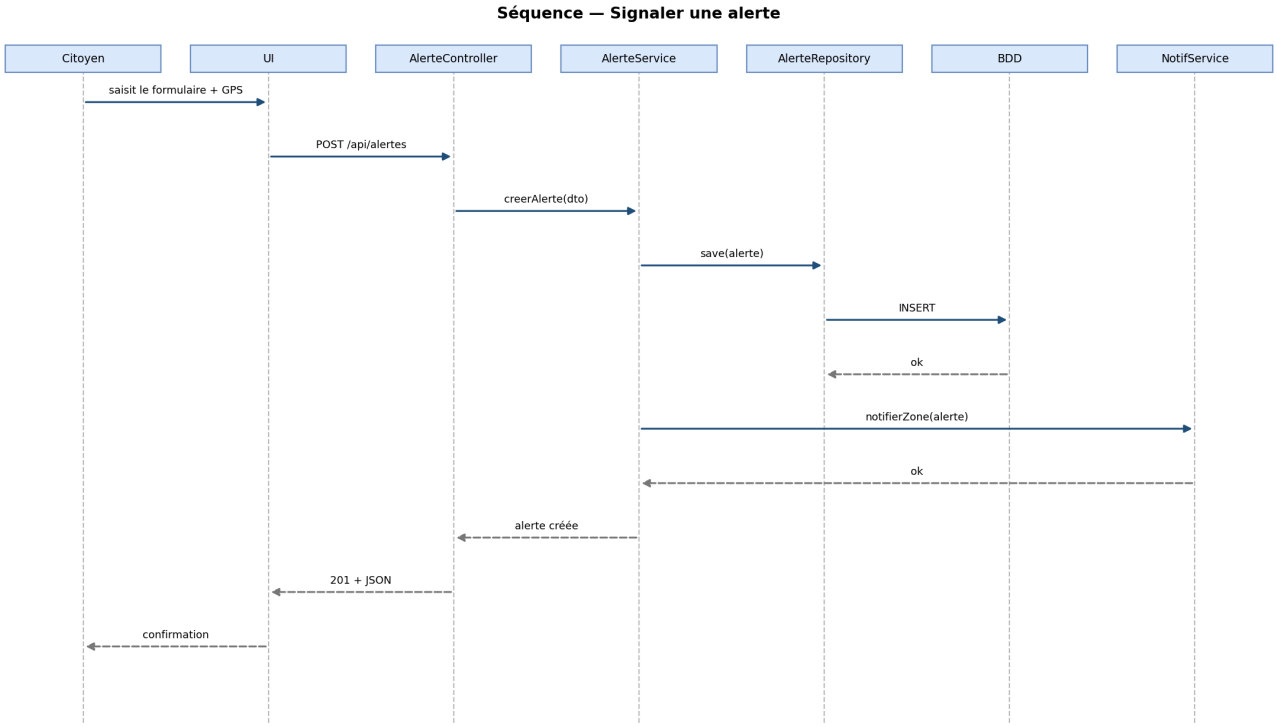
Les points les plus importants

- Les 6 entités reprennent les tables du MPD : **c'est le pont entre MERISE et UML**.
- Une **couche de services** porte la logique : AlerteService, MeteoService, NotificationService.
- **Composition** (losange plein) : une Notification n'existe pas sans son Alerte.
- **Agrégation** (losange vide) : un Utilisateur crée des Alertes qui existent indépendamment.
- Les cardinalités (1 → 0..*) rappellent les mêmes règles que le MCD.

À dire à l'oral : « Mes tables MERISE deviennent mes classes : c'est la continuité entre la base de données et le code. »

6. Diagramme de séquence — Signaler une alerte

Jalon 4 — UML · Montre l'ordre des étapes dans le temps pour une action précise.



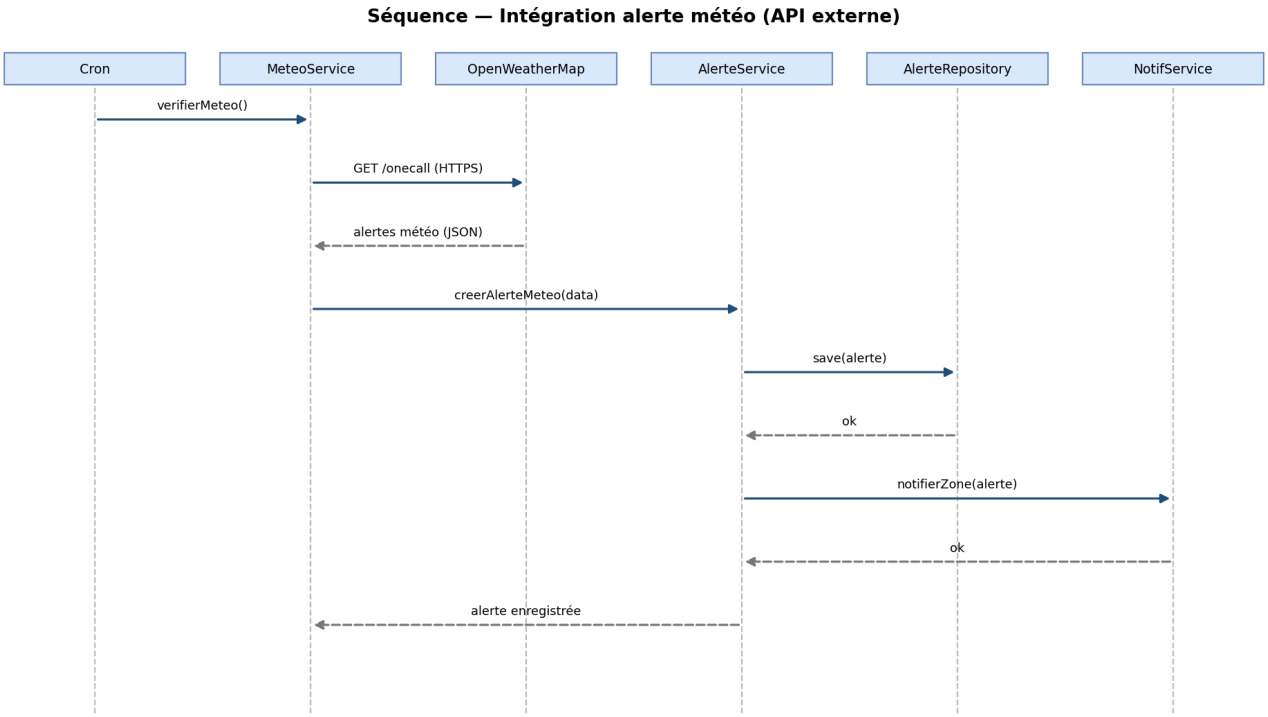
Les points les plus importants

- Le citoyen envoie le formulaire → le **Controller** reçoit la requête (POST /api/alertes).
- Le Controller appelle le **Service métier**, qui crée l'alerte.
- Le **Repository** enregistre en base (INSERT) et confirme.
- Le Service demande ensuite au **NotificationService** de prévenir les utilisateurs de la zone.
- La réponse remonte jusqu'à l'utilisateur (confirmation affichée).

À dire à l'oral : « Ce diagramme montre ma séparation en couches : Controller → Service → Repository → Base. »

7. Diagramme de séquence — Intégration météo (API externe)

Jalon 4 — UML · Montre comment l'API externe alimente l'application automatiquement.



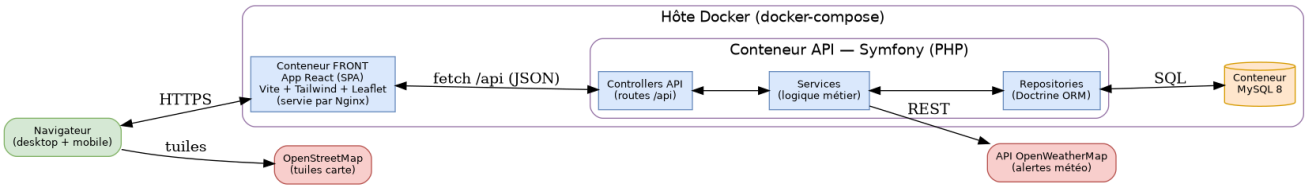
Les points les plus importants

- C'est **automatique** : un planificateur (Cron) déclenche la vérification, personne ne saisit à la main.
- Le **MeteoService** interroge OpenWeatherMap en HTTPS avec une clé API.
- Si une alerte météo est détectée, elle est **transformée en alerte AlerteZone**.
- Elle est enregistrée en base, puis les utilisateurs concernés sont notifiés.
- C'est la réponse à la contrainte « intégrer au moins une API tierce ».

À dire à l'oral : « La météo, personne ne la saisit : l'API OpenWeatherMap crée les alertes toute seule. »

8. Architecture multi-couches

Jalon 4 — Architecture · La vue d'ensemble technique : comment les briques sont organisées et déployées.



Les points les plus importants

- **Architecture séparée** : un front React (SPA) et une API Symfony, qui communiquent en JSON (fetch).
- Le back imposé reste **Symfony (PHP)** ; React est une bibliothèque JavaScript côté client, pas un remplaçant.
- Organisation en couches : Controllers API → Services → Repositories (Doctrine) → MySQL.
- Tout est **conteneurisé avec Docker** (front, API, base) : même environnement partout.
- APIs externes : OpenWeatherMap (météo) et OpenStreetMap (tuiles). CI/CD via GitHub Actions.

À dire à l'oral : « J'ai choisi l'option API Symfony + front React séparé, autorisée par le cahier des charges, pour une interface très réactive. »